Notebook 02: **Multi-Window Sequence Setup and Validation**

Purpose of This Notebook

This notebook is the first true extension notebook after the baseline lock established in Notebook 01.

That baseline notebook already did its job: it rebuilt the preliminary evidence chain in a compact final-pipeline form, verified that the dataset and label pipeline are intact, preserved the non-negotiable honesty constraints, and ended at the correct stopping point: the project is ready to move beyond the preliminary stage without rewriting history.

So this notebook must begin the extension carefully.

Its role is not to argue that pseudo-reflectivity is already better across the whole sequence. Its role is not to compare many proxy formulations. Its role is not to perform failure-case analysis. Its role is not to render videos or make downstream usefulness claims.

Its job is simpler, but foundational:

define, validate, and organize multiple contiguous frame windows across SemanticKITTI sequence 00 so that every later final-stage notebook operates on a reproducible and explicitly declared temporal structure.

In other words, this notebook is the **multi-window scaffold** for the rest of the final pipeline.

Why This Notebook Exists

The preliminary stage was intentionally local.

That was correct. The earlier work needed to answer a feasibility question first: is the reflectivity-aware augmentation idea doing anything meaningful at all?

Notebook 01 already locked the answer: yes, the project idea is real enough to continue, but the gains are modest, the proxy remains heuristic, and the results are scene-dependent rather than universal.

That means the next scientific step cannot be: repeat one favorite 10-frame or 30-frame segment more dramatically.

The next correct step is: move from a **single local temporal story** to a **structured set of multiple temporal windows**.

This notebook exists because any later claim about temporal robustness, semantic consistency across time, proxy behavior across motion, or cross-window variation becomes much stronger if it is built on several declared windows rather than one convenient hand-picked slice.

So before there are stronger metrics, richer plots, proxy comparisons, or failure analyses, there must first be a disciplined answer to this question:

> **Which exact windows of sequence 00 are we studying, how are they defined, and are they structurally valid?**

This notebook provides that answer.

Relationship to Notebook 01

Notebook 01 is the baseline lock. Notebook 02 is the first extension scaffold.

That distinction matters.

Notebook 01 established the trusted starting point:

- the SemanticKITTI loading and label pipeline works,
- a practical range-aware proxy can be computed as

$$\hat{\rho}_i = I_i \cdot R_i$$

- the proxy behaves differently from raw intensity,
- short-window motion behavior is structured,
- semantic gains can exist,
- and those gains remain modest, heuristic, and scene-dependent.

This notebook does **not** reopen those claims.

It does **not** try to improve them yet. It does **not** try to broaden them numerically yet. It does **not** mutate the baseline narrative.

Instead, it treats Notebook 01 as frozen context and asks the next engineering-scientific question:

> If later notebooks are supposed to evaluate multiple temporal regimes, then what is the actual multi-window structure of sequence 00 that they will use?

So this notebook is the first place where the final project begins to broaden in scope, but it does so only at the level of **sequence organization and validation**, not at the level of results.

Core Idea of This Notebook

The core idea is to convert sequence 00 from a long undifferentiated list of frames into a **reproducible set of contiguous analysis windows**.

Let the ordered frame IDs in sequence 00 be denoted as

$$F = \{f_0, f_1, f_2, \ldots, f_{N-1}\}$$

where N is the total number of available frames in the sequence.

A contiguous temporal window is then defined as

$$W_k^{(L)} = \{ f_{s_k}, f_{s_k+1}, \ldots, f_{s_k+L-1} \}$$

where:

- s_k is the starting frame index for window k,
- L is the window length,
- $W_k^{(L)}$ contains consecutive frames only,
- and the window is valid only if

$$s_k + L - 1 < N$$

This notebook will support multiple values of L, corresponding to different temporal scales, such as:

- **short windows**
- **medium windows**
- **long windows**

The exact chosen lengths may be decided explicitly in code, but the conceptual structure is:

$$W_{\text{short}} = \{ W_1^{(L_s)}, W_2^{(L_s)}, \ldots \}$$

$$W_{\text{medium}} = \{ W_1^{(L_m)}, W_2^{(L_m)}, \ldots \}$$

$$W_{\text{long}} = \{ W_1^{(L_\ell)}, W_2^{(L_\ell)}, \ldots \}$$

with

$$L_s < L_m < L_\ell$$

The notebook is therefore not about point-wise signal analysis yet. It is about building a clean and trustworthy temporal indexing layer over the sequence.

Scientific Motivation

A key limitation of the preliminary stage is that it relied on local windows for feasibility demonstration.

That is not a flaw. It is exactly what an early-stage study should do.

But a final-stage study should reduce narrative fragility.

If a result depends too heavily on one local segment, then there is always a risk that the chosen segment is unusually favorable, unusually stable, or unusually clean. A broader study becomes more credible when it is evaluated over multiple windows drawn from different parts of the same sequence.

So the scientific motivation of this notebook is modest but essential:

- not to prove a better proxy yet,
- not to prove semantic benefit yet,
- not to prove temporal generalization yet,

but to create the **windowed experimental substrate** on which those later tests can be run honestly.

Conceptually, later notebooks will compute some metric M over a window W_k, producing values like

$$M(W_1), M(W_2), \ldots, M(W_K)$$

and then study the distribution of those values across windows instead of over-interpreting one example.

But before that can happen, the windows themselves must be made explicit, inspectable, and valid.

This notebook is where that discipline begins.

Mathematical Setup

1. Sequence structure

Let the total ordered frame set of sequence 00 be

$$F = \{f_0, f_1, \ldots, f_{N-1}\}$$

where each frame f_t is expected to have:

- one LiDAR scan file,
- one matching semantic label file,
- and a valid position in the ordered sequence.

2. Window definition

For a chosen start index s_k and window length L, define

$$W_k^{(L)} = \{f_t \mid t \in [s_k, s_k + L - 1)\}$$

subject to the validity condition

$$0 \le s_k \le N - L$$

3. Contiguity condition

A valid analysis window must be contiguous in frame index. If frame indices are stored in sorted order, this means the selected entries must occupy consecutive positions in the ordered list.

Operationally, for a frame-index sequence

$$(i_0, i_1, \ldots, i_{L-1})$$

we require

$$i_{j+1} = i_j + 1 \text{ for all } j = 0, 1, \ldots, L-2$$

4. Label-availability condition

For each frame $f_t \in W_k^{(L)}$, there must exist a matching label file.

Define an indicator function

$$1_{\text{label}}(f_t) = \begin{pmatrix} 1, & \text{if the matching label file exists} \\ 0, & \text{otherwise} \end{pmatrix}$$

Then a structurally valid window must satisfy

$$\sum_{f_t \in W_k^{(L)}} 1_{\text{label}}(f_t) = L$$

5. Window metadata model

Each window should be representable as a metadata record of the form

$$\text{WindowMeta}_k = (\text{window_id}, \text{scale}, s_k, e_k, L, \text{first_frame_id}, \text{last_frame_id}, \text{is_valid})$$

where:

- s_k = start index.
- $e_k = s_k + L - 1$ = end index.
- L = window length.
- `scale` indicates short / medium / long.
- `is_valid` records whether structural checks passed.

This converts sequence slicing from ad hoc notebook behavior into an explicit experiment object.

What This Notebook Will Actually Do

This notebook should remain disciplined and narrow.

Step A: Locate sequence 00 and inspect its structure

We will first verify the existence of the sequence root and the expected substructure, including:

- `velodyne/`
- `labels/`
- `calib.txt`
- `poses.txt`
- `times.txt`

The goal here is not to reload heavy signal logic from Notebook 01. It is simply to confirm that the sequence used for multi-window construction is structurally intact.

Step B: Enumerate all available frame IDs

We will list and sort the available frame IDs from sequence 00, then verify:

- total frame count,
- naming consistency,
- and matching `.bin` / `.label` alignment at the sequence level.

This converts sequence 00 into an indexed ordered frame list suitable for window construction.

Step C: Define candidate temporal scales

We will explicitly declare window lengths for different temporal regimes, for example:

- short windows,
- medium windows,
- long windows.

The exact lengths are an experiment-design choice, but the notebook should make them explicit and reusable.

Step D: Choose several contiguous windows across the sequence

We will select windows from different parts of sequence 00 rather than clustering everything around the beginning.

The exact selection strategy may be fixed manually, evenly spaced, or guided by simple coverage logic, but it must remain:

- transparent,
- reproducible,
- and easy to audit.

Step E: Validate every chosen window

For every window, we will verify:

- index bounds,
- contiguity,
- frame count,
- first and last frame IDs,
- and label availability for every included frame.

If any window fails validation, that must be visible in the notebook rather than hidden.

Step F: Build clean metadata tables

This notebook should end with organized tables such as:

- one row per window,
- one row per frame-within-window if needed,
- scale labels,
- start/end indices,
- start/end frame IDs,
- length,
- validation flags.

Step G: Export metadata for later notebooks

The validated metadata should be saved in compact reproducible form, such as CSV files, so later notebooks can consume the same exact windows without redefining them from scratch.

This matters because later notebooks should compare results, not compare different accidental window definitions.

What This Notebook Is Not Allowed to Do

This needs to stay explicit because Notebook 02 sits at a dangerous transition point: it is easy for a setup notebook to get overexcited and start sneaking in claims that belong later.

This notebook is **not** the place to:

- compare I, $I \cdot R$, $I \cdot R^2$, log variants, clipped variants, or normalized variants,
- compute semantic separability gains across many windows,
- rank proxies,
- identify best or worst cases,
- perform class-wise gain analysis,
- build failure-case narratives,
- export videos or GIFs,
- claim sequence-wide semantic benefit,
- claim temporal robustness of any proxy across the whole sequence,
- or test downstream usefulness.

All of that belongs later.

This notebook is a **window-definition notebook**, not a results notebook. Its contribution is experimental structure, not scientific drama.

Engineering Motivation

A long sequence becomes messy very quickly if every later notebook chooses its own local slice ad hoc.

Without a standardized window layer, the pipeline risks:

- inconsistent frame ranges across notebooks,
- accidental duplication of windows,

- silent mismatch between plots and summaries,
- irreproducible temporal comparisons,
- and subtle narrative drift.

So this notebook is also an engineering cleanup step.

It creates a shared temporal contract for the rest of `final_pipeline/`.

Later notebooks should be able to say, in effect:

> use the validated window metadata from Notebook 02.

instead of:

> trust me bro, I sliced some frames around here somewhere.

That tiny difference saves a shocking amount of future chaos.

Expected Outputs

By the end of this notebook, we should have:

1. a verified sequence-00 frame inventory,
2. a declared set of short, medium, and long window lengths,
3. several reproducible contiguous windows drawn from different parts of the sequence,
4. per-window validation checks showing that the selected frames and labels exist,
5. a clean window summary table,
6. optional per-frame membership tables for each window,
7. metadata CSV files that later notebooks can load directly,
8. and one explicit concluding statement that the final pipeline is no longer tied to a single local segment.

These outputs are intentionally infrastructural. They are not yet scientific result tables.

Expected Outcome and Correct Interpretation

If this notebook succeeds, the correct conclusion will be something like this:

- sequence 00 has been converted into a validated multi-window analysis structure,
- the final-stage pipeline now has declared temporal coverage at multiple scales,
- later notebooks can perform cross-window comparisons on a reproducible basis,
- and the project is no longer bottlenecked by one hand-picked short segment.

That is enough. That is the win here.

Anything beyond that would be premature.

This notebook should not end by claiming that pseudo-reflectivity already wins across those windows. It should only establish that the windows are real, valid, and ready for later analysis.

Core Honesty Statement for Notebook 02

This notebook does not strengthen the reflectivity claim directly. It strengthens the **evaluation structure**.

The project still studies a reflectivity-aware augmentation idea, not calibrated reflectivity recovery.

The proxy used in the broader project remains:

$$\hat{\rho}_i = I_i \cdot R_i$$

but this notebook is not evaluating that proxy in depth yet. It is only preparing the multi-window temporal substrate on which later evaluations can be performed honestly.

So the honesty conditions remain active here too:

- no calibrated reflectivity claim,
- no material-identification claim,
- no universal-improvement claim,
- no premature sequence-wide conclusion.

The only claim this notebook should earn is:

> the multi-window experimental setup for sequence 00 has been defined and validated cleanly enough for the next stage of the final pipeline.

Expected Final Takeaway of This Notebook

Notebook 01 said: the baseline is real, compact, and locked.

Notebook 02 should now say: the multi-window study structure is defined, validated, and ready.

That is the correct handoff.

Not proxy wars yet. Not semantic heatmaps yet. Not failure-case drama yet. Not shiny video propaganda yet.

Just one careful notebook that turns sequence 00 into a trustworthy set of contiguous short, medium, and long analysis windows so the actual final-stage evaluation can begin without temporal ambiguity.

Locate and Verify Sequence 00 Structure

Before defining any temporal windows, we must confirm that the dataset path and sequence structure are correctly accessible from this notebook.

Even though Notebook 01 already verified the dataset, this step is repeated in a minimal form here to ensure that Notebook 02 is self-contained and does not silently depend on prior notebook state.

In this step, we:

- define candidate dataset root paths,
- resolve the correct existing path,
- check for required sequence components:
  - velodyne/
  - labels/
  - calib.txt
  - poses.txt
  - times.txt

This is a structural sanity check only.

We are **not loading any point clouds yet**. We are **not selecting frames yet**.

The goal is simple:

> confirm that sequence 00 exists and is complete before building window definitions on top of it.

```
from pathlib import Path

# Candidate dataset roots (same philosophy as Notebook 01)
candidate_roots = [
    Path("../data/semantickitti_subset/dataset/sequences/00"),
    Path("../data/semantickitti/dataset/sequences/00"),
]

# Resolve dataset root
DATASET_ROOT = next((p for p in candidate_roots if p.exists()),
candidate_roots[0])

# Expected structure
expected_items = [
    "velodyne",
    "labels",
    "calib.txt",
    "poses.txt",
    "times.txt",
]

print("Resolved Dataset Root:")
print(DATASET_ROOT.resolve())
print("Exists:", DATASET_ROOT.exists())
print()

print("Checking expected items:")
for item in expected_items:
    item_path = DATASET_ROOT / item
    print(f"{item:10} -> exists: {item_path.exists()} | path:
```

```
{item_path}")

print()

if DATASET_ROOT.exists():
    print("Sequence directory contents:")
    for p in sorted(DATASET_ROOT.iterdir()):
        print(" -", p.name)
else:
    print("WARNING: Dataset root not found. Checked paths:")
    for p in candidate_roots:
        print(" -", p.resolve())
```

```
Resolved Dataset Root:
/home/twilightpriest/GitHub/reflect-aug-seg/data/semantickitti/dataset
/sequences/00
Exists: True

Checking expected items:
velodyne   -> exists: True | path:
../data/semantickitti/dataset/sequences/00/velodyne
labels     -> exists: True | path:
../data/semantickitti/dataset/sequences/00/labels
calib.txt  -> exists: True | path:
../data/semantickitti/dataset/sequences/00/calib.txt
poses.txt  -> exists: True | path:
../data/semantickitti/dataset/sequences/00/poses.txt
times.txt  -> exists: True | path:
../data/semantickitti/dataset/sequences/00/times.txt

Sequence directory contents:
 - calib.txt
 - labels
 - poses.txt
 - times.txt
 - velodyne
```

Enumerate and Validate Frame Inventory

Now that the sequence structure is confirmed, we move one level deeper: we extract the full list of available frame IDs from the sequence.

This step converts the dataset from a directory structure into an ordered temporal index that can be used for window construction.

In this step, we:

- list all `.bin` files from the velodyne directory,
- list all `.label` files from the labels directory,
- sort both lists to enforce temporal ordering,

- verify that:
  - both lists have the same length,
  - frame IDs match exactly between point clouds and labels.

This gives us a globally consistent frame index:

$$F = \{f_0, f_1, \ldots, f_{N-1}\}$$

This is a critical invariant: every frame used in later window definitions must exist in both the LiDAR stream and the label stream.

We are still **not loading any point clouds**.

We are only building a clean, verified temporal index.

```
# Define directories
velodyne_dir = DATASET_ROOT / "velodyne"
labels_dir = DATASET_ROOT / "labels"

# Collect files
velodyne_files = sorted(velodyne_dir.glob("*.bin"))
label_files = sorted(labels_dir.glob("*.label"))

# Extract frame IDs (stems)
velodyne_ids = [f.stem for f in velodyne_files]
label_ids = [f.stem for f in label_files]

print("Frame Inventory Summary")
print()
print("Total velodyne frames :", len(velodyne_ids))
print("Total label frames    :", len(label_ids))
print()

# Show samples
print("First 5 frame IDs:")
print(velodyne_ids[:5])
print()

print("Last 5 frame IDs:")
print(velodyne_ids[-5:])
print()

# Consistency checks
counts_match = len(velodyne_ids) == len(label_ids)
ids_match = velodyne_ids == label_ids

print("Consistency Checks")
print()
print("Counts match :", counts_match)
print("Frame IDs match exactly :", ids_match)
```

```python
# Hard assertion (this is important)
assert counts_match, "Mismatch in number of .bin and .label files"
assert ids_match, "Frame ID mismatch between velodyne and labels"

print("\nGlobal frame alignment check PASSED.")

# Store for later use (important)
FRAME_IDS = velodyne_ids
NUM_FRAMES = len(FRAME_IDS)

print("\nDerived Sequence Parameters")
print()
print("Total frames (N):", NUM_FRAMES)
print("First frame ID  :", FRAME_IDS[0])
print("Last frame ID   :", FRAME_IDS[-1])
```

```
Frame Inventory Summary

Total velodyne frames : 4541
Total label frames    : 4541

First 5 frame IDs:
['000000', '000001', '000002', '000003', '000004']

Last 5 frame IDs:
['004536', '004537', '004538', '004539', '004540']

Consistency Checks

Counts match : True
Frame IDs match exactly : True

Global frame alignment check PASSED.

Derived Sequence Parameters

Total frames (N): 4541
First frame ID  : 000000
Last frame ID   : 004540
```

Define Temporal Window Scales

With the full sequence index available, we now define the temporal scales that will be used throughout the final pipeline.

At this stage, we are not selecting specific windows yet. We are only defining the **allowed window lengths** that represent different temporal regimes.

We introduce three categories:

- short windows

- medium windows
- long windows

Each category corresponds to a fixed number of consecutive frames:

$$L_s < L_m < L_\ell$$

These lengths should satisfy two constraints:

1. They must be large enough to capture meaningful temporal variation.
2. They must be small enough to fit within the total sequence length (N).

These values are not arbitrary: they represent different observational scales of motion and scene evolution.

This step establishes a consistent temporal vocabulary for the rest of the pipeline.

Later notebooks will refer to these scales instead of redefining window lengths ad hoc.

We will also validate that each chosen window length is feasible:

$$L \leq N$$

so that valid windows can be constructed without boundary violations.

```
# Define window lengths (you can tweak later, but keep structure
consistent)
WINDOW_SCALES = {
    "short": 10,
    "medium": 30,
    "long": 100,
}

print("Defined Window Scales")
print()
for scale, length in WINDOW_SCALES.items():
    print(f"{scale:6} -> {length} frames")

print()

# Validate feasibility against sequence length
print("Feasibility Check")
print()
for scale, length in WINDOW_SCALES.items():
    is_valid = length <= NUM_FRAMES
    print(f"{scale:6} -> length={length}, valid={is_valid}")
    assert is_valid, f"{scale} window length exceeds total frame
count"

print("\nAll window scales are valid for this sequence.")
```

```
# Store for later use (important)
WINDOW_LENGTHS = WINDOW_SCALES

Defined Window Scales

short  -> 10 frames
medium -> 30 frames
long   -> 100 frames

Feasibility Check

short  -> length=10, valid=True
medium -> length=30, valid=True
long   -> length=100, valid=True

All window scales are valid for this sequence.
```

Define Window Start Indices Across the Sequence

We now select starting indices for windows across the full sequence.

The goal is to avoid bias toward a single local segment (e.g., always starting at frame 0) and instead cover different parts of the sequence.

Strategy:

- For each window scale (short, medium, long), we compute a set of start indices.
- These start indices are **evenly spaced** across the valid index range:

$$s \in [0, N - L)$$

- We choose a fixed number of windows per scale (e.g., 5) to balance coverage and computational load.

For each scale:

- define maximum valid start index:

$$s_{max} = N - L$$

- sample evenly spaced start indices:

$$s_k = \text{linspace}(0, s_{max}, K)$$

This produces:

- diverse temporal coverage,
- reproducible window selection,
- no manual cherry-picking.

We do **not** build frame lists yet. We only compute and validate the start indices.

```python
import numpy as np

# Number of windows per scale (can adjust later if needed)
NUM_WINDOWS_PER_SCALE = 5

WINDOW_STARTS = {}

print("Window Start Index Selection")
print()

for scale, length in WINDOW_LENGTHS.items():
    max_start = NUM_FRAMES - length

    # Evenly spaced start indices
    starts = np.linspace(0, max_start, NUM_WINDOWS_PER_SCALE,
dtype=int)

    WINDOW_STARTS[scale] = starts.tolist()

    print(f"\nScale: {scale}")
    print(f" - window length : {length}")
    print(f" - max start idx : {max_start}")
    print(f" - selected starts: {starts.tolist()}")

    # Sanity checks
    assert all(0 <= s <= max_start for s in starts), "Start index out
of bounds"

print("\nAll window start indices are valid.")

# Store for later
WINDOW_START_INDICES = WINDOW_STARTS
```

```
Window Start Index Selection


Scale: short
 - window length : 10
 - max start idx : 4531
 - selected starts: [0, 1132, 2265, 3398, 4531]

Scale: medium
 - window length : 30
 - max start idx : 4511
 - selected starts: [0, 1127, 2255, 3383, 4511]

Scale: long
 - window length : 100
 - max start idx : 4441
 - selected starts: [0, 1110, 2220, 3330, 4441]
```

```
All window start indices are valid.
```

Construct Full Frame Windows from Start Indices

We now convert the validated start indices into actual contiguous frame windows.

For each scale and each start index s_k, we construct:

$$W_k^{(L)} = \{f_{s_k}, f_{s_k+1}, \ldots, f_{s_k+L-1}\}$$

This step transforms abstract indices into explicit frame ID sequences.

For each window, we will:

- extract the corresponding slice from the global frame list,
- verify that:
  - the number of frames equals the expected window length L,
  - the frames are contiguous in index space,
- store the window in a structured format.

The output of this step is a dictionary:

$$\text{WINDOWS}[\text{scale}] = [W_1, W_2, \ldots, W_K]$$

where each W_k is a list of frame IDs.

This is the first point where the temporal structure becomes directly usable by later notebooks.

```
WINDOWS = {}

print("Constructing Frame Windows")
print()

for scale, starts in WINDOW_START_INDICES.items():
    length = WINDOW_LENGTHS[scale]
    scale_windows = []

    print(f"\nScale: {scale}")
    print(f" - window length: {length}")

    for i, start_idx in enumerate(starts):
        end_idx = start_idx + length

        # Slice frame IDs
        window_frames = FRAME_IDS[start_idx:end_idx]

        # Sanity checks
        assert len(window_frames) == length, "Window length mismatch"
```

```
        # Contiguity check (index-based)
        indices = list(range(start_idx, end_idx))
        reconstructed_ids = [FRAME_IDS[idx] for idx in indices]
        assert window_frames == reconstructed_ids, "Non-contiguous
window detected"

        scale_windows.append({
            "window_id": f"{scale}_{i}",
            "start_idx": start_idx,
            "end_idx": end_idx - 1,
            "frame_ids": window_frames,
        })

        print(f"   window {i}: start={start_idx}, end={end_idx-1}")

    WINDOWS[scale] = scale_windows

print("\nAll windows constructed successfully.")

# Store for later
WINDOW_DEFINITIONS = WINDOWS
```

```
Constructing Frame Windows


Scale: short
 - window length: 10
   window 0: start=0, end=9
   window 1: start=1132, end=1141
   window 2: start=2265, end=2274
   window 3: start=3398, end=3407
   window 4: start=4531, end=4540

Scale: medium
 - window length: 30
   window 0: start=0, end=29
   window 1: start=1127, end=1156
   window 2: start=2255, end=2284
   window 3: start=3383, end=3412
   window 4: start=4511, end=4540

Scale: long
 - window length: 100
   window 0: start=0, end=99
   window 1: start=1110, end=1209
   window 2: start=2220, end=2319
   window 3: start=3330, end=3429
   window 4: start=4441, end=4540

All windows constructed successfully.
```

Build Window Metadata Table

We now convert the constructed window definitions into a structured tabular format.

So far, windows are stored as nested dictionaries. This is useful for computation, but not ideal for inspection, validation, or reuse.

In this step, we:

- flatten all windows across all scales,
- create one row per window,
- store key metadata:
  - window_id.
  - scale (short / medium / long).
  - start index.
  - end index.
  - window length.
  - first frame ID.
  - last frame ID.

This produces a DataFrame:

$$\text{WindowMeta} = \left[\left(\text{window_id}, \text{scale}, s_k, e_k, L, f_{start}, f_{end}\right)\right]$$

This table will serve as the canonical reference for all later notebooks.

We are still not doing any signal or semantic analysis. This is purely structural organization.

```
import pandas as pd

metadata_rows = []

for scale, windows in WINDOW_DEFINITIONS.items():
    length = WINDOW_LENGTHS[scale]

    for w in windows:
        row = {
            "window_id": w["window_id"],
            "scale": scale,
            "start_idx": w["start_idx"],
            "end_idx": w["end_idx"],
            "length": length,
            "first_frame": w["frame_ids"][0],
            "last_frame": w["frame_ids"][-1],
        }
        metadata_rows.append(row)

window_df = pd.DataFrame(metadata_rows)
```

```python
print("Window Metadata Table")
print()
print("Shape:", window_df.shape)
print()

print(window_df)
```

```
Window Metadata Table

Shape: (15, 7)

   window_id   scale  start_idx  end_idx  length first_frame
last_frame
0    short_0   short          0        9      10      000000
000009
1    short_1   short       1132     1141      10      001132
001141
2    short_2   short       2265     2274      10      002265
002274
3    short_3   short       3398     3407      10      003398
003407
4    short_4   short       4531     4540      10      004531
004540
5   medium_0  medium          0       29      30      000000
000029
6   medium_1  medium       1127     1156      30      001127
001156
7   medium_2  medium       2255     2284      30      002255
002284
8   medium_3  medium       3383     3412      30      003383
003412
9   medium_4  medium       4511     4540      30      004511
004540
10    long_0    long          0       99     100      000000
000099
11    long_1    long       1110     1209     100      001110
001209
12    long_2    long       2220     2319     100      002220
002319
13    long_3    long       3330     3429     100      003330
003429
14    long_4    long       4441     4540     100      004441
004540
```

Export Window Metadata for Reuse

We now export the window metadata table to a CSV file.

This ensures that all later notebooks in the final pipeline use the exact same window definitions without recomputing them.

This step is important for:

- reproducibility,
- consistency across notebooks,
- avoiding accidental redefinition of windows.

The exported file becomes the single source of truth for temporal window structure.

No further computation is performed here.

```
from pathlib import Path

# Create output directory (if not exists)
output_dir = Path("../results/window_metadata")
output_dir.mkdir(parents=True, exist_ok=True)

# Save CSV
output_path = output_dir / "sequence00_windows.csv"
window_df.to_csv(output_path, index=False)

print("Window metadata exported successfully.")
print("Path:", output_path.resolve())
```

```
Window metadata exported successfully.
Path:
/home/twilightpriest/GitHub/reflect-aug-seg/results/window_metadata/
sequence00_windows.csv
```

Final Conclusion

In this notebook, sequence 00 has been converted into a structured, multi-window temporal dataset.

We defined short, medium, and long window scales, selected evenly distributed start indices across the full sequence, and constructed fully validated contiguous frame windows with complete label availability.

The result is a clean and reproducible window metadata table that serves as the temporal backbone for all subsequent analysis.

At this stage, no signal-level or semantic claims are made.

The pipeline is now ready to move from: **“which frames to analyze”** to: **“what those frames reveal.”**